VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors - Microcontrollers

Report lab 2

Timer Interrupt and LED Scanning

Advisor(s): Phan Van Sy
Student(s): Tran Le Dang Khoi 2452599

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Exercise 1 **5**
1.1 Requirement . . . 5
1.2 Circuit Schematic . . . 5
1.3 Source Code . . . 5

2 Exercise 2 **7**
2.1 Requirement . . . 7
2.2 Circuit Schematic . . . 7
2.3 Source Code . . . 7

3 Exercise 3 **8**
3.1 Requirement . . . 8
3.2 Source Code . . . 9

4 Exercise 4 **10**
4.1 Requirement . . . 10
4.2 Source Code . . . 10

5 Exercise 5 **11**
5.1 Requirement . . . 11
5.2 Source Code . . . 11

6 Exercise 6 **12**
6.1 Requirement . . . 12
6.2 Source Code . . . 12
6.3 Report Questions . . . 13

7 Exercise 7 **13**
7.1 Requirement . . . 13
7.2 Source Code . . . 14

8 Exercise 8 **15**
8.1 Requirement . . . 15
8.2 Source Code . . . 15

9 Exercise 9 **17**
9.1 Requirement 17
9.2 Circuit Schematic 17
9.3 Source Code 18

10 Exercise 10 **19**
10.1 Requirement 19
10.2 Source Code 20

List of Figures

List of Tables

Listings

1 Exercise 1

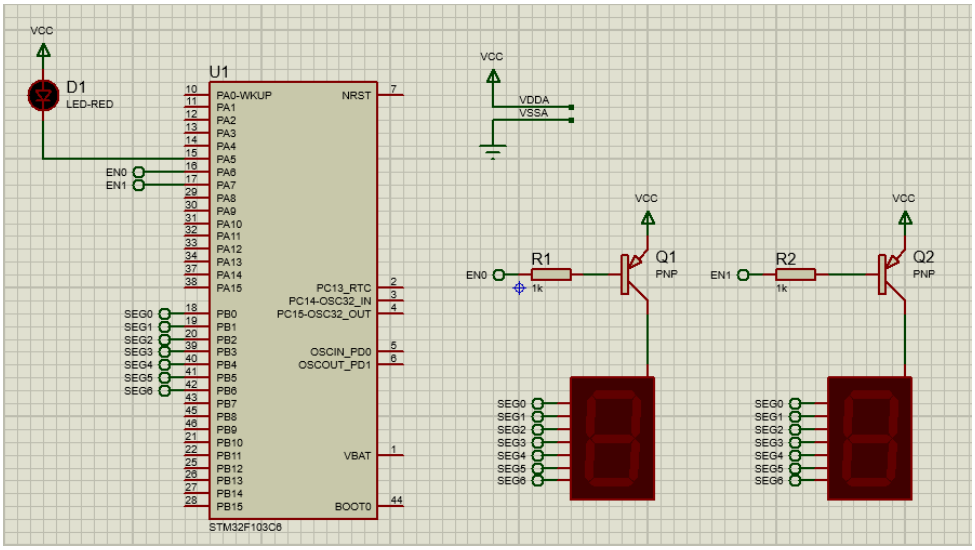
1.1 Requirement

The first exercise uses two common-anode seven-segment displays with shared segment signals connected to PB0–PB6 of the STM32F103C6. Each display is enabled through a PNP transistor. The first display shows digit 1, while the second displays digit 2. The two displays are selected alternately every 500 ms using a TIM2 interrupt.

Since the displays are common-anode, the segment signals are active-low. The display-enable signals on PA6 and PA7 are also active-low. TIM2 is configured to generate an interrupt every 10 ms.

1.2 Circuit Schematic

Proteus schematic for two multiplexed seven-segment displays.



Source here

1.3 Source Code

The `display7SEG(int num)` function from Lab 1 is reused. The following callback changes the active display after 50 TIM2 interrupts.

```
uint8_t counter = 0;
uint8_t index_led = 0;

```

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  counter++;

  if(counter >= 50)
  {
    counter = 0;

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);

    if(index_led == 0)
    {
      display7SEG(1);
      HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
      index_led = 1;
    }
    else if (index_led == 1)
    {
      display7SEG(2);
      HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
      index_led = 0;
    }
  }
}
```

The interrupt is started using `HAL_TIM_Base_Start_IT(&htim2)`. The switching rate is 2 selections per second. A complete scan of both displays takes 1 second, corresponding to a full-scan frequency of 1 Hz.

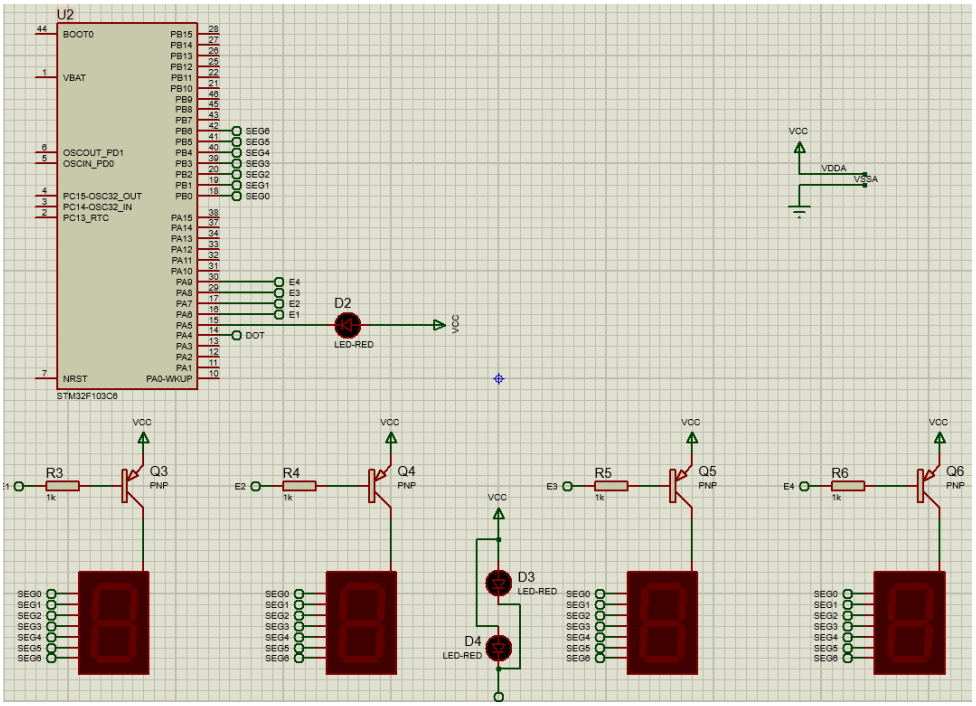
2 Exercise 2

2.1 Requirement

Exercise 2 extends the circuit to four common-anode seven-segment displays and a pair of DOT LEDs controlled by PA4. The display shows `12:30`, with one digit enabled at a time. Each digit remains selected for 500 ms, while the DOT changes state every 500 ms to create a blinking effect with a 1-second period.

2.2 Circuit Schematic

Proteus schematic for four seven-segment displays and the DOT LEDs. The four enable signals are connected to PA6–PA9.



Source here

2.3 Source Code

```
const uint16_t enPins[4] = {
    GPIO_PIN_6, GPIO_PIN_7, GPIO_PIN_8, GPIO_PIN_9
};
const int digits[4] = {1, 2, 3, 0};
```

```
int index_led = 0;
int scan_tick = 0, dot_tick = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance != TIM2) return;

    if (++scan_tick >= 50)
    {
        scan_tick = 0;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7 |
                                 GPIO_PIN_8 | GPIO_PIN_9,
                                    GPIO_PIN_SET);
        display7SEG(digits[index_led]);
        HAL_GPIO_WritePin(GPIOA, enPins[index_led],
           GPIO_PIN_RESET);
        index_led = (index_led + 1) % 4;
    }

    if (++dot_tick >= 50)
    {
        dot_tick = 0;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }
}
```

One complete scan takes $4 \times 500 = 2000$ ms. Therefore, the full-scan frequency is 0.5 Hz.

3 Exercise 3

3.1 Requirement

Exercise 3 introduces the `led_buffer[4]` array and the `update7SEG(int index)` function. Each array element contains the digit to be displayed at the corresponding position. The function turns off all displays before changing the segment signals and en-

abling the selected digit. This allows the displayed values to be updated without changing the scanning logic.

3.2 Source Code

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};

void update7SEG(int index)
{
    const uint16_t enPins[4] = {
        GPIO_PIN_6, GPIO_PIN_7, GPIO_PIN_8, GPIO_PIN_9
    };

    if (index < 0 || index >= MAX_LED) return;

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7 |
                             GPIO_PIN_8 | GPIO_PIN_9,
                               GPIO_PIN_SET);
    display7SEG(led_buffer[index]);
    HAL_GPIO_WritePin(GPIOA, enPins[index], GPIO_PIN_RESET);
}
```

The function is called from the TIM2 interrupt as follows:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance != TIM2) return;

    if (++scan_tick >= 50)
    {
        scan_tick = 0;
        update7SEG(index_led);
        index_led = (index_led + 1) % MAX_LED;
    }

```

```
    if (++dot_tick >= 50)
    {
        dot_tick = 0;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }
}
```

4 Exercise 4

4.1 Requirement

Exercise 4 changes the scanning period so that all four seven-segment displays are scanned once every second. Each digit is therefore selected for 250 ms. The DOT continues blinking with a period of one second.

4.2 Source Code

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance != TIM2) return;

    if (++scan_tick >= 25)
    {
        scan_tick = 0;
        update7SEG(index_led);
        index_led = (index_led + 1) % MAX_LED;
    }

    if (++dot_tick >= 50)
    {
        dot_tick = 0;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
    }
}
```

With a 10 ms timer interrupt, 25 interrupts correspond to 250 ms per digit. The full-scan frequency is therefore 1 Hz.

5 Exercise 5

5.1 Requirement

Exercise 5 implements a digital clock that displays the current hour and minute on four seven-segment displays. The clock starts at 15:08:50 and updates the seconds every 1000 ms. When seconds reach 60, minutes are incremented. Hours are updated when minutes reach 60, and the hour value returns to 0 after 23.

The `updateClockBuffer()` function converts the hour and minute values into four decimal digits. A leading zero is displayed when a value contains only one digit.

5.2 Source Code

```
int hour = 15, minute = 8, second = 50;
int led_buffer[4] = {1, 5, 0, 8};

void updateClockBuffer(void)
{
    led_buffer[0] = hour / 10;
    led_buffer[1] = hour % 10;
    led_buffer[2] = minute / 10;
    led_buffer[3] = minute % 10;
}
```

The clock is updated in the main loop. The display-scanning code from Exercise 4 is retained.

```
while (1)
{
    HAL_Delay(1000);
    second++;
    if (second >= 60) { second = 0; minute++; }
    if (minute >= 60) { minute = 0; hour++; }
    if (hour >= 24) hour = 0;
```

```
    updateClockBuffer();
}
```

6 Exercise 6

6.1 Requirement

Exercise 6 introduces a software timer to replace blocking delays. TIM2 generates an interrupt every 10 ms. Each interrupt decreases the software-timer counter, and a flag is raised when the requested interval has elapsed. The main loop checks this flag to perform the required operation without using `HAL_Delay()`.

6.2 Source Code

```
volatile int timer0_counter = 0;
volatile int timer0_flag = 0;
int TIMER_CYCLE = 10;

void setTimer0(int duration)
{
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}

void timer_run(void)
{
    if (timer0_counter > 0)
    {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
```

```
22     if (htim->Instance == TIM2) timer_run();
23 }
```

The software timer is tested by toggling the red LED on PA5:

```
1 setTimer0(1000);
2 HAL_TIM_Base_Start_IT(&htim2);
3
4 while (1)
5 {
6     if (timer0_flag == 1)
7     {
8         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
9         setTimer0(2000);
10    }
11 }
```

6.3 Report Questions

1. **What happens if `setTimer0(1000)` is omitted?** The counter and flag remain zero. Since `timer_run()` only decreases a positive counter, the flag is never raised and the LED does not toggle.

2. **What happens if it is replaced by `setTimer0(1)`?** With a 10 ms timer cycle, integer division gives $1/10 = 0$. The counter does not start, so the LED does not toggle.

3. **What happens if it is replaced by `setTimer0(10)`?** The counter is initialized to one tick. The first toggle occurs after approximately 10 ms. Subsequent toggles occur every 2000 ms, because the timer is restarted using `setTimer0(2000)`.

7 Exercise 7

7.1 Requirement

Exercise 7 updates the digital clock using software timers instead of `HAL_Delay()`. The clock advances every 1000 ms, and the DOT on PA4 blinks independently. Both

operations are handled in the main loop, while the seven-segment scanning from Exercise 5 is retained.

7.2 Source Code

The clock uses `timer0` and the DOT uses a second independent software timer, `timer2`, which is included in `timer_run()`.

```
setTimer0(1000);
setTimer2(500);
HAL_TIM_Base_Start_IT(&htim2);

while (1)
{
    if (timer0_flag == 1)
    {
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24) hour = 0;

        updateClockBuffer();
        setTimer0(1000);
    }

    if (timer2_flag == 1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        setTimer2(500);
    }
}
```

8 Exercise 8

8.1 Requirement

Exercise 8 also moves `update7SEG()` from the TIM2 interrupt callback to the main loop. A separate software timer triggers the scanning of four digits every 250 ms. After this modification, the interrupt callback only updates software timers, while the main loop handles the clock, DOT, and seven-segment displays.

8.2 Source Code

The final program uses three software timers: `timer0` for the clock, `timer1` for seven-segment scanning, and `timer2` for the DOT. They are processed by the same `timer_run()` function.

```
void setTimer1(int duration)
{
    timer1_counter = duration / TIMER_CYCLE;
    timer1_flag = 0;
}

void setTimer2(int duration)
{
    timer2_counter = duration / TIMER_CYCLE;
    timer2_flag = 0;
}

void timer_run(void)
{
    if (timer0_counter > 0)
    {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
    if (timer1_counter > 0)
    {
        timer1_counter--;
```

```
        if (timer1_counter == 0) timer1_flag = 1;
    }
    if (timer2_counter > 0)
    {
        timer2_counter--;
        if (timer2_counter == 0) timer2_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) timer_run();
}
```

```
updateClockBuffer();
update7SEG(0);
index_led = 1;
setTimer0(1000);
setTimer1(250);
setTimer2(500);
HAL_TIM_Base_Start_IT(&htim2);

while (1)
{
    if (timer0_flag == 1)
    {
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24) hour = 0;
        updateClockBuffer();
        setTimer0(1000);
    }

    if (timer1_flag == 1)
    {
```

```
23          update7SEG(index_led);
24          index_led = (index_led + 1) % MAX_LED;
25          setTimer1(250);
26      }
27
28      if (timer2_flag == 1)
29      {
30          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
31          setTimer2(500);
32      }
33  }
```

9 Exercise 9

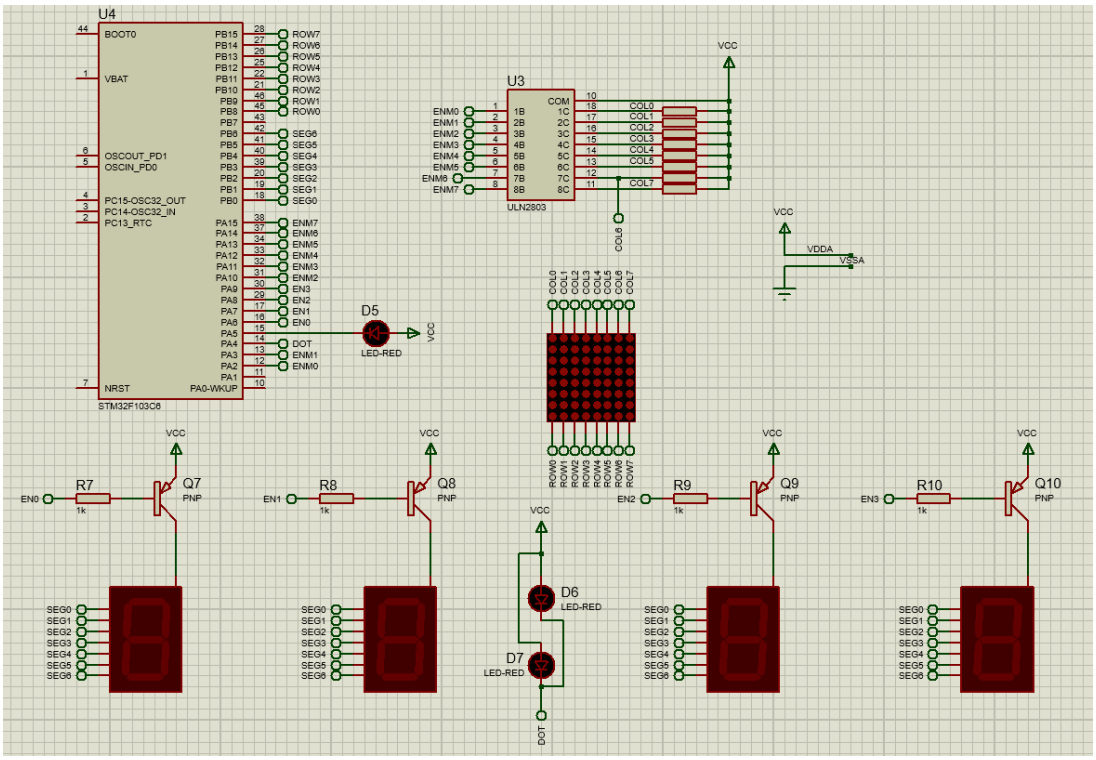
9.1 Requirement

Exercise 9 adds a `MATRIX-8X8-RED` LED matrix and a `ULN2803` transistor array to the existing digital clock. Eight row signals are connected to PB8–PB15. The column-enable signals `ENM0-ENM7` are connected to PA2, PA3, and PA10–PA15.

The `updateLEDMatrix(int index)` function scans one column at a time to display the character A. In the tested Proteus circuit, both row selection and column enabling use active-low logic.

9.2 Circuit Schematic

Proteus schematic for the digital clock with the LED matrix and ULN2803.



Source here

9.3 Source Code

```
const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;
uint8_t matrix_buffer[8] = {
    0x00, 0x7C, 0x12, 0x11,
    0x11, 0x12, 0x7C, 0x00
};

void updateLEDMatrix(int index)
{
    const uint16_t enPins[8] = {
        GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_10, GPIO_PIN_11,
        GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
    };
    const uint16_t rowPins[8] = {
        GPIO_PIN_8, GPIO_PIN_9, GPIO_PIN_10, GPIO_PIN_11,
```

```
        GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
    };

    if (index < 0 || index >= MAX_LED_MATRIX) return;

    for (int i = 0; i < 8; i++)
        HAL_GPIO_WritePin(GPIOA, enPins[i], GPIO_PIN_SET);

    for (int i = 0; i < 8; i++)
        HAL_GPIO_WritePin(GPIOB, rowPins[i],
            (matrix_buffer[index] & (1 << i)) ?
            GPIO_PIN_RESET : GPIO_PIN_SET);

    HAL_GPIO_WritePin(GPIOA, enPins[index], GPIO_PIN_RESET);
}
```

The matrix is initialized with all rows and column-enable signals HIGH. The display is scanned from the main loop using a 2 ms interval.

```
if (HAL_GetTick() - matrix_tick >= 2)
{
    matrix_tick = HAL_GetTick();
    updateLEDMatrix(index_led_matrix);
    index_led_matrix++;
    if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix
       = 0;
}
```

Each complete eight-column scan takes approximately 16 ms, corresponding to a target refresh rate of 62.5 Hz.

10 Exercise 10

10.1 Requirement

Exercise 10 implements a left-shifting animation on the LED matrix. The character A starts outside the right edge, gradually enters the display, moves across it, and disappears

at the left edge. The animation then repeats. The seven-segment clock and DOT continue operating independently.

The animation updates the matrix data every 150 ms. Column scanning remains at 2 ms per column.

10.2 Source Code

The original character pattern is stored separately from the display buffer. The buffer is initially empty so that the character does not appear at the center when the simulation starts.

```
const uint8_t letterA[6] = {
    0x7C, 0x12, 0x11, 0x11, 0x12, 0x7C
};
uint8_t matrix_buffer[8] = {0};
int animation_pos = 8;
uint32_t animation_tick = 0;

void shiftMatrixLeft(void)
{
    animation_pos--;
    if (animation_pos < -6) animation_pos = 8;

    for (int i = 0; i < 8; i++)
        matrix_buffer[i] = 0;

    for (int i = 0; i < 6; i++)
    {
        int col = animation_pos + i;
        if (col >= 0 && col < 8)
            matrix_buffer[col] = letterA[i];
    }
}
```

`animation_tick` is initialized with `HAL_GetTick()` before entering the main loop. The following block is added to the loop from Exercise 9:

```
if (HAL_GetTick() - animation_tick >= 150)
```



```
{
    animation_tick = HAL_GetTick();
    shiftMatrixLeft();
}
```

The function `shiftMatrixLeft()` modifies only the contents of `matrix_buffer`. The function `updateLEDMatrix()` from Exercise 9 continues scanning the updated buffer, so the animation does not require a blocking delay or an additional hardware timer.

GitHub Repository

All Proteus schematics and related project files for Lab02 are available at:
https://github.com/Kaytii210/Microprocessors-Microcontrollers/tree/main/Lab02